Designing the System

Snapchat Masking & Hacking

Zoe J.D.M. Spyrou
Student Number: 452280
Fontys HBO-ICT
Semester 4 OL

Table of Contents

- Snapchat Masking & Hacking
- Edit Log
- Context
- Objective
- Scope
- Research Methodology
- Stakeholder Requirements
  - Functional Requirements
  - Non-Functional Requirements
- Infrastructure Requirements Analysis
  - Service Level Agreements SLA
  - Security Requirements
  - Automation Requirements
- Technical Design
  - Architecture Overview
  - Infrastructure Components
    - Application Packaging
    - Python GUI Application
    - Media Processing
    - ADB Integration
    - Android VCAM Module
- Automation Strategy
  - Build and Packaging
  - Deployment
- Security Design
  - Defense in Depth
- Support
- Conclusion
- Sources

Edit Log

Date	Task
21/1/2026	Finished first version
23/1/2026	Added sources

Introduction

The police uses Snapchat to talk to suspected individuals, through fake/controlled accounts portraying to be underage children, using AI generated images. The goal of our project is to find a way to mask the activity of the police and extract the IP address of the suspects, so that they can then use it to find out the location of the suspect.

Masking is defined as:

- Make it appear as if messages sent on Snapchat are being sent on a mobile device, not a browser, as Snapchat notifies users from where messages are being sent through
- Images that are being sent need to appear as images that were taken through Snapchat, and not as images directly uploaded through a camera roll, as those appear different in Snapchat
- Making an account look relatively active (snapscore not 0, aka not 0 messages sent in total, as it appears that the account has not been used yet)
- Not notifying the suspect of screenshots taken of the conversation, needed for collection of evidence

All research and testing is done on accounts that are either owned by us (the team that is working on the project) or the stakeholders (police Rotterdam, Fontys coaches). No non-consenting parties are used in testing/experimenting. These efforts are not to be used in any scope outside the project, as that goes against the policy of Snapchat. All evidence found is to be kept between the Stakeholders and us.

Context

This document describes the design process of the system infrastructure and explains how stakeholder requirements were translated into a concrete technical solution.

Objective

Based on communication with the stakeholder, functional and non-functional requirements are transformed into a tangible, deployable system design.

Scope

This document covers the complete communication flow between system components from an infrastructure perspective. It includes requirements analysis, design decisions, automation strategy, security considerations, deployment, and maintenance.

Research Methodology

Based on the DOT Framework, the following methodologies were applied:

Method	What / How / Why
Literature Study	Research best practices for infrastructure design, security guidelines, and media processing standards.
Interviews	Stakeholder interviews to gather functional and non-functional infrastructure requirements.
Best, Good, and Bad Practices	Analysis of industry standards for deployment, automation, and security.
Prototyping	Validation of technical feasibility through proof-of-concept implementations.

Stakeholder Requirements

The stakeholder provided high-level requirements. Through interviews, demonstrations, and follow-up communication, the following requirements were identified.

Functional Requirements

1. Images and videos must be sent on Snapchat without a media upload tag.
2. Support for commonly used image and video formats.
3. Ability to capture screenshots or recordings without notifying the sender.

Non-Functional Requirements

1. Usability: Accessible to non-technical users.
2. Portability: Easy to duplicate and run on different computers.
3. Platform Compatibility: Must support Windows.
4. Documentation: Clear end-user documentation.
5. Performance: Minimal latency in media processing and delivery.

Infrastructure Requirements Analysis

Service Level Agreements (SLA)

Aspect	Target	Rationale
Availability	95% during active usage	User-initiated system
Response Time	Less than 1 minute media processing	Acceptable for interactive usage
Recovery Time	Less than 5 minutes	Simple restart for non-technical users
Support	Documentation-based	Self-service support model

Security Requirements

- Local-only processing to prevent data leakage.

- Minimal permissions for all components.
- USB-based communication only.

Automation Requirements

- Standalone executable distribution.
- One-click deployment.
- Bundled dependencies such as FFmpeg and ADB.
- Separate builds for Windows and Linux (production and development)

Technical Design

Architecture Overview

The system is designed as a desktop application, prioritizing simplicity and usability.

Infrastructure Components

Application Packaging

Technology: PyInstaller

Rationale:

- Single executable distribution for Windows and Linux.
- No Python, Docker, or dependency installation required.
- Native USB and display access.
- Reduced operational complexity.

Docker was considered but rejected due to display forwarding complexity, USB passthrough requirements, additional setup steps, and increased difficulty for non-technical users, as well as technical users. With PyInstaller as the main packaging application, the user simply clicks the app and is ready to start using it.

Python GUI Application

Technology: CustomTkinter

Purpose:

- Media selection and upload.
- Device selection and management.

- Embedded screen streaming.

Windows forms using the .NET framework was considered, but rejected due to the fact that the stakeholder is then limited to the Windows operating system, or at least make it very hard to switch over to Linux, should they choose to do so. Moreover, data handling is a lot easier to manage through Python, and it easier for the stakeholder to debug any events, should he wish to continue the project.

What was also considered was a browser based UI, as that would make the Docker display forwarding issue non-existent. It still left the USB passthrough issue though, and data would be streamed with latency. Port forwarding would also have to be set up, increasing complexity.

Media Processing

Technology: FFmpeg (bundled)

Functionality:

- Image and video format conversion.
- Real-time decoding.
- Screen stream rendering.

Instead of integrating a scrcpy window, based on a local install of scrcpy, the choice was made to use a scrcpy server, on the Android device. This TCP stream is then decoded within the GUI using FFmpeg. It was also considered to completely scrap the scrcpy integration and rely purely on the raw h264 stream from the mobile device, using adb's screenrecorder functionality, but translating that stream was proven unreliable and caused extreme delay in receiving and decoding the frames.

ADB Integration

Technology: Bundled ADB platform tools

Functionality:

- USB-based device communication.
- File transfer.

ADB was used as the basis of communicating with the mobile device. There were no alternatives considered, as this is the standard method of communication with Android devices.

Android VCAM Module

Implementation: LSPosed module

Functionality:

- Overrides the camera feed in Snapchat.
- Replaces live camera stream with uploaded media.
- Requires a rooted Android device.

Prior to finally settling on a custom LSPosed module that works on the hardware level of the device, software based solutions, such as Snapenhanced were considered. Software based solutions are ineffective when trying to modify any sort of behavior on a production grade application, let alone Snapchat, that constantly releases new versions and even pays people to find bugs in their software. The only solution was to focus on something that Snapchat could not control, the hardware level API's, such as Camera2.

Automation Strategy

Build and Packaging

- PyInstaller specification defines the complete build process.
- Platform-specific binaries for Windows and Linux.
- Automated CI/CD pipeline for building the executables, no pushing of full binaries.

Deployment

1. Download the executable for the appropriate operating system.
2. Connect the Android device via USB.
3. Launch the executable.

No installation or administrator privileges are required after initial setup.

Security Design

Defense in Depth

Network Layer:

- USB-only ADB communication. All data is sent directly from and to the phone with serial connection, nothing is streamed over the internet.

- No exposed network ports.
- No internet dependency. The GUI can send over media to the Android phone through serial connection.

Application Layer:

- Application runs with standard user permissions. No need for administrator rights to run it.
- Input validation for media files. The program cannot crash, it gracefully handles wrong input type. Moreover, there is support for many of the most famous types, such as jpeg, png, tiff, mp4, webp, etc.
- Cleanup of temporary files. No files are stored anywhere else on the device besides their original location after closing the application.

Device Layer:

- Root access using Magisk and LSPosed.
- Restricted file system access. The only folder with write permissions is the folder the media is being written to.
- Fixed Device Integrity with modules further explained in the Technical Guide. This means that any apps can be downloaded and used on the phone, without the Play Store flagging the device as unsafe.

Support

---

1. User manual and technical manual, for instructions and troubleshooting.
2. Direct line of communication with developer.

Conclusion

This infrastructure design translates stakeholder requirements into a secure, portable, and user-friendly system. By prioritizing simplicity and usability, the design replaces container-based complexity with a standalone executable approach that better suits non-technical users.

Sources

Most information mentioned in the above document comes from prior research, further elaborated on in the following documents:
Desktop GUI and Cross-Platform Streaming

- ADB, screenrecord, python and multithreading, PyAV and FFmpeg, GUI apps in Docker, Docker image for building the Windows GUI
  Replacing the Android Phone's Camera With a Virtual Camera
- Camera2, Hardware Abstraction Layer, Xposed Module Repository
  Translating the solution to a user-friendly end product
- Scrcpy client/server, tkinter and custom tkinter, more of FFmpeg

Honorable mentions:

Technical Guide

- All info on installation of the environment, in depth, with testing on 2 mobile devices and 2 different OS's, Windows and Linux

User Guide

- All information on using the desktop GUI

Sirion. (2025, December 29). *What is an SLA? The Key to Better Business Relationships*. Sirion. https://www.sirion.ai/library/contracts/service-level-agreement-sla/